

Chapter 1

1.What is OOP in java. Explain the properties of OOP.

Object-Oriented Programming (OOP) in Java is a programming paradigm that uses "objects" to represent data and methods. Java is a fully object-oriented language (except for primitives like int, char, etc.) and supports all main features of OOP, allowing developers to build modular, flexible, and reusable software systems.

Properties of OOP

❖ Encapsulation

- Hides data using private variables.
- Accessed via get and set methods.

❖ Inheritance

- One class inherits from another using extends.
- Promotes code reuse.

❖ Polymorphism

- Same method behaves differently (overloading/overriding).
- Achieved via method overriding and interfaces.

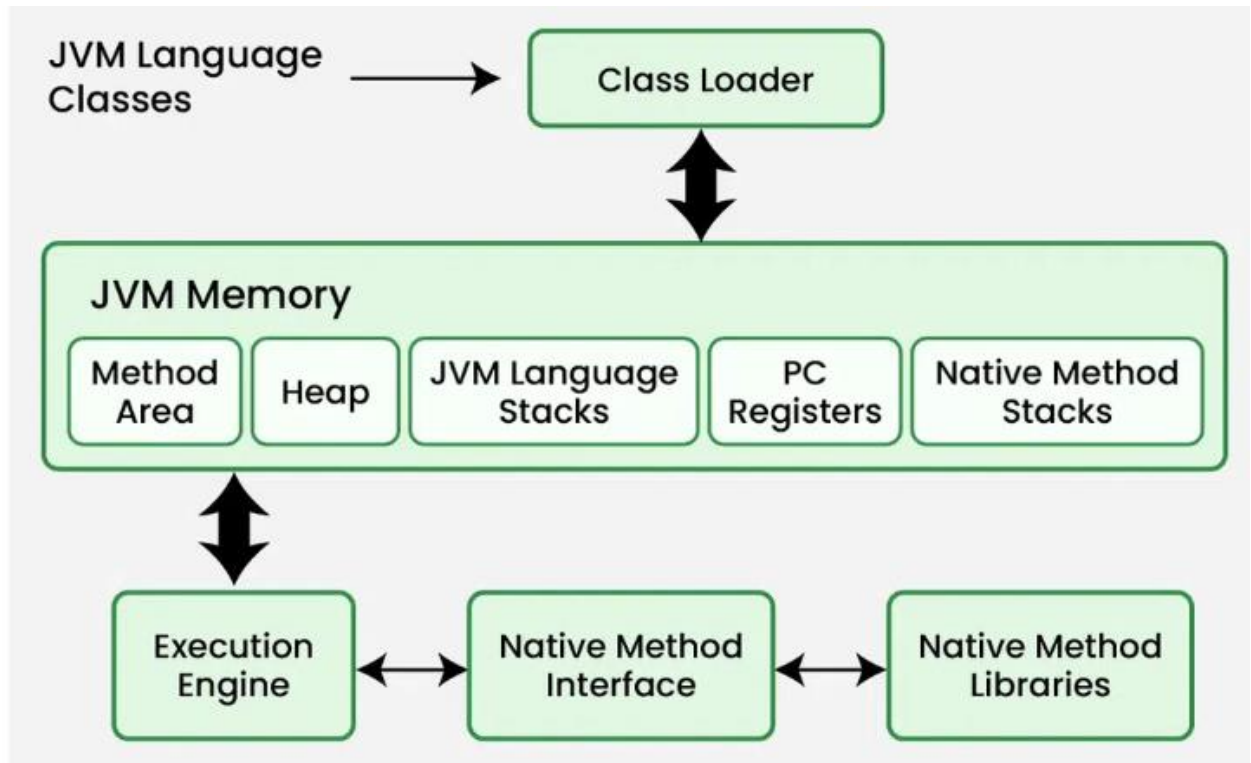
❖ Abstraction

- Hides complex details, shows only essentials.
- Achieved using abstract classes or interfaces.

2.What is JVM. Explain with complete diagram.

JVM(Java Virtual Machine) runs Java applications as a run-time engine. JVM is the one that calls the **main** method present in a Java code.

Diagram of JVM



3.What is operator. Explain different operator in java with example.

operators are **special symbols that perform operations on variables or values**. These operators are essential in programming as they allow you to manipulate data efficiently.

Types of operator in java

1. Arithmetic Operators

[Arithmetic Operators](#) are used to perform simple arithmetic operations on primitive and non-primitive data types.

- * : Multiplication
- / : Division
- % : Modulo
- + : Addition
- - : Subtraction

2. Unary Operators

[Unary Operators](#) need only one operand. They are used to increment, decrement, or negate a value.

- - , Negates the value.
- + , Indicates a positive value (automatically converts byte, char, or short to int).
- ++ , Increments by 1.

- **Post-Increment:** Uses value first, then increments.
 - **Pre-Increment:** Increments first, then uses value.
- -- , Decrements by 1.
 - **Post-Decrement:** Uses value first, then decrements.
 - **Pre-Decrement:** Decrements first, then uses value.
- ! , Inverts a Boolean value.

3. Assignment Operator

‘=’ Assignment operator is used to assign a value to any variable.

Common compound operators include:

- += , Add and assign.
- -= , Subtract and assign.
- *= , Multiply and assign.
- /= , Divide and assign.
- %= , Modulo and assign.

4. Relational Operators

Relational Operators are used to check for relations like equality, greater than, and less than.

Relational operators compare values and return Boolean results:

- == , Equal to.
- != , Not equal to.
- < , Less than.
- <= , Less than or equal to.
- > , Greater than.
- >= , Greater than or equal to.

5. Logical Operators

Logical Operators are used to perform “logical AND” and “logical OR” operations, similar to AND gate and OR gate in digital electronics.

Conditional operators are:

- **&&, Logical AND:** returns true when both conditions are true.
- **||, Logical OR:** returns true if at least one condition is true.
- **!, Logical NOT:** returns true when a condition is false and vice-versa

6. Bitwise Operators

Bitwise Operators are used to perform the manipulation of individual bits of a number and with any of the integer types.

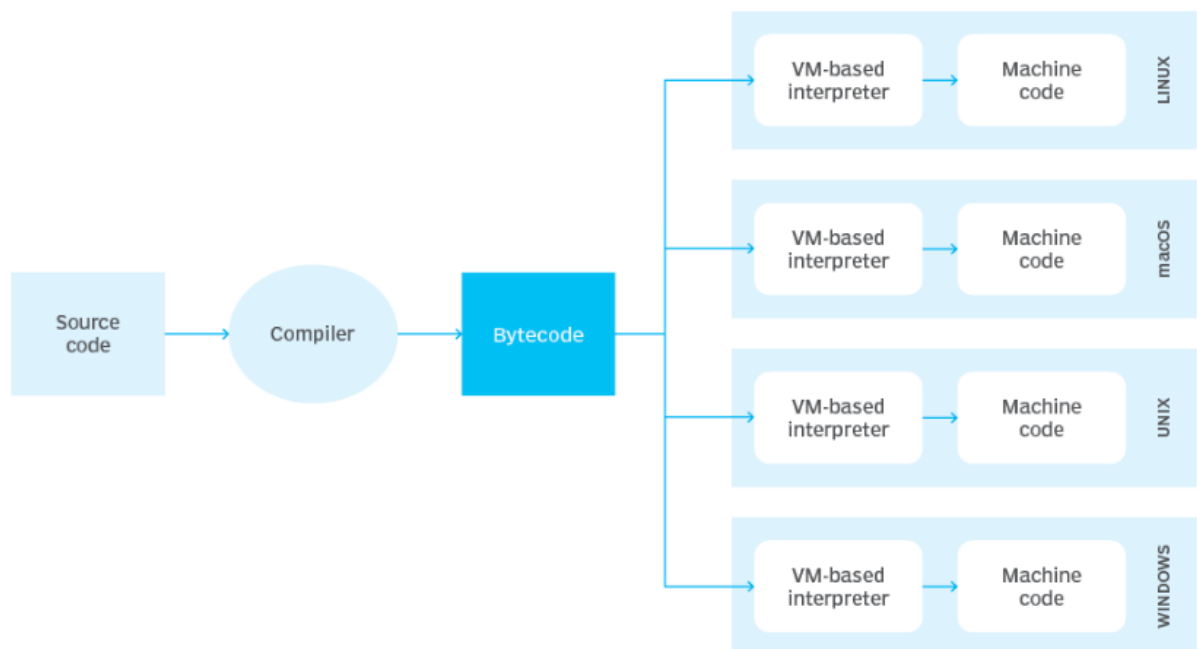
some bitwise operator are:

- **& (Bitwise AND)**: returns bit-by-bit AND of input values.
- **| (Bitwise OR)**: returns bit-by-bit OR of input values.
- **^ (Bitwise XOR)**: returns bit-by-bit XOR of input values.
- **~ (Bitwise Complement)**: inverts all bits (one's complement).

4.What is Byte code.

Byte Code can be defined as an intermediate code generated by the compiler after the compilation of source code(JAVA Program). This intermediate code makes Java a platform-independent language.

Bytecode in application delivery



5.What is flowchart. Explain with diagram.

A flowchart is a visual representation of a process, system, or algorithm, using symbols connected by arrows to depict the flow of steps or actions.

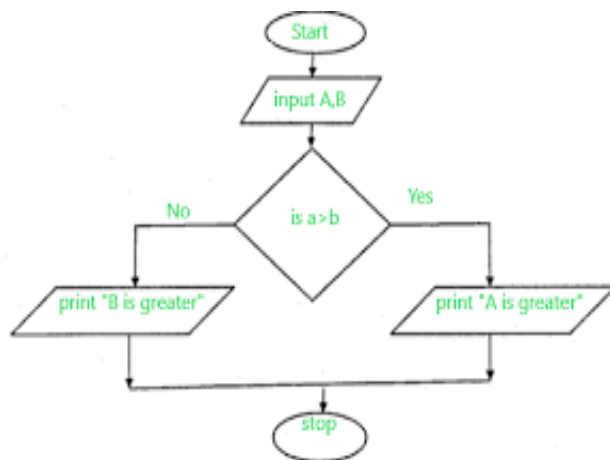


Fig: flowchart diagram

Chapter 2

1.What is datatype. Explain different datatype?

Data types in Java are of different sizes and values that can be stored in a variable that is made as per convenience and circumstances to cover up all test cases.

Java has two categories in which data types are segregated

- **Primitive Data Type:** These are the basic building blocks that store simple values directly in memory. Examples of primitive data types are **boolean, char, byte, short, int, long, float, and double.**
- **Non-Primitive Data Types (Object Types):** These are reference types that store memory addresses of objects. Examples of Non-primitive data types are String, Array, Class, Interface, and Object

2.Write a program to find the day of week using switch.

```

public class DayOfWeek {
    public static void main(String[] args) {
        int day = 4; // Example day number (1 for Monday, 2 for Tuesday, etc.)
        String dayName;
    }
}
  
```

```
switch (day) {  
    case 1:  
        dayName = "Monday";  
        break;  
    case 2:  
        dayName = "Tuesday";  
        break;  
    case 3:  
        dayName = "Wednesday";  
        break;  
    case 4:  
        dayName = "Thursday";  
        break;  
    case 5:  
        dayName = "Friday";  
        break;  
    case 6:  
        dayName = "Saturday";  
        break;  
    case 7:  
        dayName = "Sunday";  
        break;  
    default:  
        dayName = "Invalid day";  
}  
System.out.println(dayName); // Output the day name  
}  
}
```

3. What is loop?(while, do-while, for).

A loop in Java is a control flow statement that allows a block of code to be executed repeatedly based on a condition.

There are three main types of loops in Java: while, do-while, and for.

- while loop: The while loop evaluates a condition before executing the code block. If the condition is true, the code block is executed, and the condition is evaluated again. This process repeats until the condition becomes false.

Syntax:

```
int i = 0;
while (i < 5) {
    System.out.println(i);
    i++;
}
```

- do-while loop: The do-while loop is similar to the while loop, but it evaluates the condition after executing the code block. This means that the code block will always be executed at least once, even if the condition is false.

Syntax:

```
int i = 0;
do {
    System.out.println(i);
    i++;
} while (i < 5);
```

- for loop: The for loop is used to execute a block of code a specific number of times. It consists of three parts: initialization, condition, and update.

Syntax:

```
for (int i = 0; i < 5; i++) {
    System.out.println(i);
}
```

4. WAP to find .

a) Factorial

Syntax:

```
import java.util.Scanner;
```

```
public class Factorial {
```

```
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = scanner.nextInt();
```

```

long factorial = 1;

if (num < 0) {
    System.out.println("Factorial is not defined for negative numbers.");
} else {
    for (int i = 1; i <= num; i++) {
        factorial *= i;
    }
    System.out.println("Factorial of " + num + " is: " + factorial);
}
scanner.close();
}
}

```

b) Fibonacci series(n th term)

Syntax:

```

public class Fibonacci{
    // Function to calculate the nth Fibonacci number using
    // recursion
    static int nthFibonacci(int n){
        // Base case: if n is 0 or 1, return n
        if (n <= 1) {
            return n;
        }
        // Recursive case: sum of the two preceding
        // Fibonacci numbers
        return nthFibonacci(n - 1) + nthFibonacci(n - 2);
    }

    public static void main(String[] args){
        int n = 5;
        int result = nthFibonacci(n);
        System.out.println(result);
    }
}

```

5. What is break statement and continue statement?

Break and continue statements are control flow tools used within loops (for, while, do-while) and switch statements to alter the normal execution flow.

- **Break Statement:** The break statement terminates the innermost loop or switch statement immediately. Execution resumes at the statement following the terminated structure.

Syntax:

```
for (int i = 0; i < 10; i++) {  
    if (i == 5) {  
        break; // Exit the loop when i equals 5  
    }  
    System.out.println(i);  
}  
// Output: 0 1 2 3 4
```

- **Continue Statement:** The continue statement skips the rest of the current iteration of a loop. The loop then proceeds to the next iteration, if any.

Syntax:

```
for (int i = 0; i < 10; i++) {  
    if (i % 2 == 0) {  
        continue; // Skip even numbers  
    }  
    System.out.println(i);  
}  
// Output: 1 3 5 7 9
```

Chapter-3

1. What is “This” keyword? Explain.

The this keyword in Java serves as a reference to the current object within an instance method or constructor.

Syntax:

```
public class Main {  
  
    int x;
```

```
// Constructor with a parameter
public Main(int x) {
    this.x = x;
}

// Call the constructor
public static void main(String[] args) {
    Main myObj = new Main(5);
    System.out.println("Value of x = " + myObj.x);
}
}
```

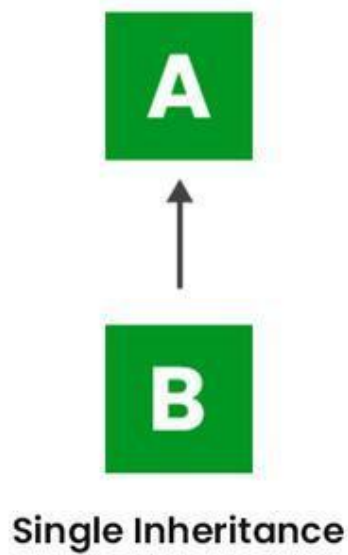
2. What is inheritance. Explain its type.

Inheritance is implemented using the **extends** keyword. The class that inherits is called the **subclass (child class)**, and the class being inherited from is called the **superclass (parent class)**.

Types of inheritance

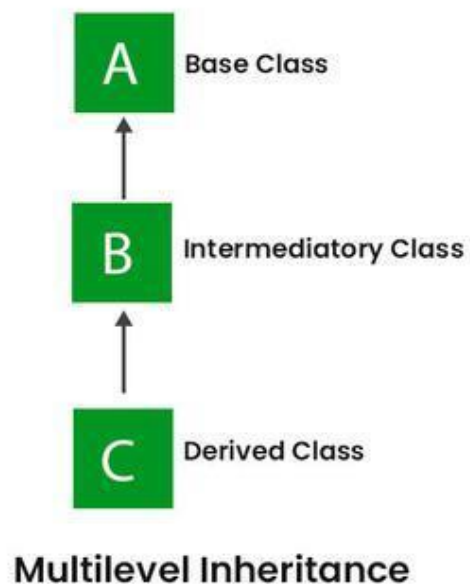
1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



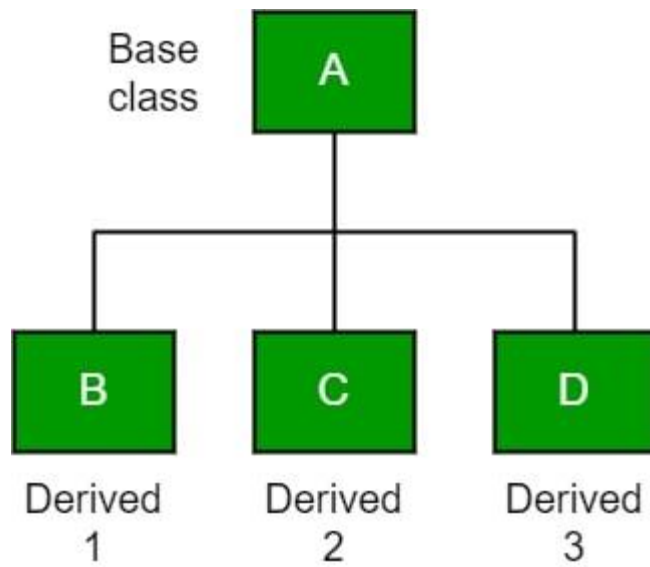
2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes.



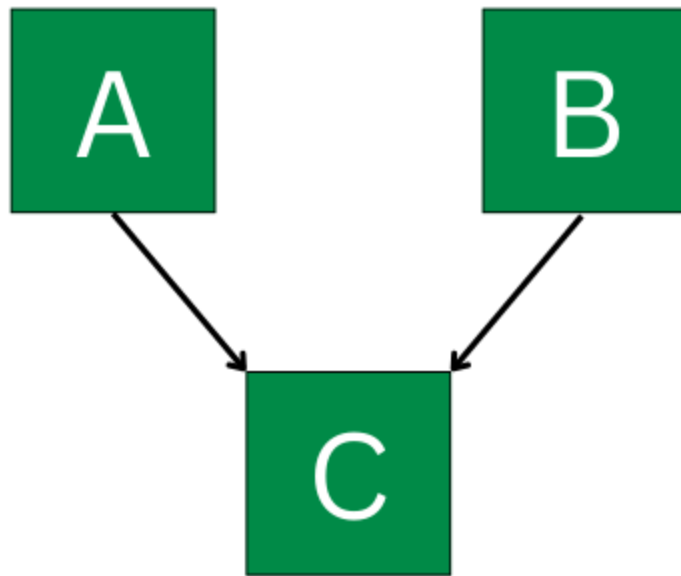
3. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.



4. Multiple Inheritance (Through Interfaces)

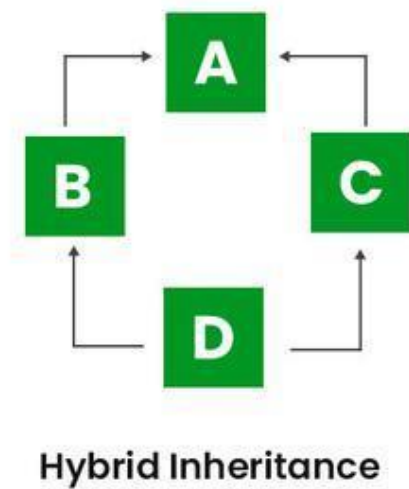
In [Multiple inheritances](#), one class can have more than one superclass and inherit features from all parent classes.



**Multiple
Inheritance**

5. Hybrid Inheritance

It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritances with classes, hybrid inheritance involving multiple inheritance is also not possible with classes.



3. What is constructor? Explain.

A constructor is a special block of code that is called when an object is created.

For example:

```
// Java Program to demonstrate
// Constructor usage
import java.io.*;

// Driver Class
class Constructor{

    // Constructor
    constructor()
    {
        super();
        System.out.println("Constructor Called");
    }
}
```

```

    }

    // main function
    public static void main(String[] args)
    {
        Constructor geek = new Constructor();
    }
}

```

4. What is overriding and overloading?

Overloading: It happens within the same class. Multiple methods can have the same name but different parameters (number, type, or order).

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

```

Overriding: It happens between a superclass and a subclass. A subclass provides a specific implementation for a method already defined in its superclass.

```

class Animal {
    void makeSound() {
        System.out.println("Generic animal sound");
    }
}

```

```
}
```

```
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Woof!");  
    }  
}
```

5. What is Recursion? Explain with example.

Recursion is a process in which a function calls itself directly or indirectly is called recursion and the corresponding function is called a recursive function.

For example:

```
int fact(int n)  
{  
    if (n <= 1) // base case  
        return 1;  
    else  
        return n*fact(n-1);  
}
```

6.What is inner class?

=> An inner class is a class inside another class.

-It is used to group classes logically, and it can access the members (even private) of the outer class.

Example:

```
// program that uses an inner class
```

```
class Outer {  
    class Inner {  
        void display() {  
            System.out.println("This is inner class");  
        }  
    }  
}
```

7. What is Abstraction. Explain with example?

=> Abstraction means hiding the internal details and showing only the essential features.

- It helps to focus on what an object does, not how it does it.

Example using Abstract Class:

```
abstract class Animal {  
    abstract void sound(); // abstract method  
}
```

```
class Dog extends Animal {  
    void sound() {
```

```
        System.out.println("Dog barks");
    }
}
```

8. What is method. Explain its types?

=> A method is a block of code that performs a specific task.

- It runs only when it is called, and helps in code reuse.

It's types:

1. Argument and return

- A method can take arguments (inputs) and can return a value after processing.

Example:

```
class Calculator {

    int add(int a, int b) {

        return a + b;

    }


    public static void main(String[] args) {

        Calculator c = new Calculator();

        int result = c.add(5, 3);

        System.out.println("Sum: " + result);

    }

}
```

2. Argument and no return

- This type of method takes arguments but does not return any value.
- It usually performs an action like printing or updating data

Example:

```
class Calculator {  
    void add(int a, int b) {  
        int sum = a + b;  
        System.out.println("Sum: " + sum);  
    }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        c.add(10, 5); // passing arguments  
    }  
}
```

3.No Argument and Return

- This type of method does not take any arguments but returns a value after performing some task.

Example:

```
class Calculator {  
    int getPi() {  
        return 3; // returning the value  
    }  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
        int piValue = c.getPi(); // calling method and storing returned value  
        System.out.println("Value of Pi: " + piValue);  
    }  
}
```

```
}  
}
```

4.No Argument No Return

- This type of method does not take any arguments and does not return a value. It simply performs an action, such as printing a message.

Example:

```
class Greet {  
    void sayHello() {  
        System.out.println("Hello, welcome!");  
    }  
  
    public static void main(String[] args) {  
        Greet g = new Greet();  
        g.sayHello(); // calling the method  
    }  
}
```

9.What is access control?

=> Access control in Java determines the visibility and accessibility of classes, methods, and variables. It is controlled using access modifiers.

Access Modifiers in java:

- 1.public: Accessible from anywhere (inside or outside the class).
- 2.private: Accessible only within the same class.

3.protected: Accessible within the same package or in subclasses (even outside the package).

Example:

```
class MyClass {  
    public int x = 10;  // accessible everywhere  
    private int y = 20;  // accessible only within MyClass  
    protected int z = 30;  // accessible within same package or subclasses  
}
```

10. What is Encapsulation?

=> Encapsulation is the process of hiding data (variables) and protecting it by wrapping it inside methods (in a class).

- It allows controlled access to the data using getters and setters.

Example:

```
class Person {  
    private String name;  // private variable  
    public void setName(String n) {  // setter  
        name = n;  
    }  
  
    public String getName() {  // getter  
        return name;  
    }  
}
```

Chapter 4

Q.N. 1) What is inheritance? Explain different types of inheritance with its block diagram.

-> Inheritance is the mechanism in Java by which one class is allowed to inherit the features (fields and methods) of another class. In Java, Inheritance means creating new classes based on existing ones. In Java, inheritance is implemented using the extends keyword. The class that inherits is called the subclass (child class), and the class being inherited from is called the superclass (parent class).

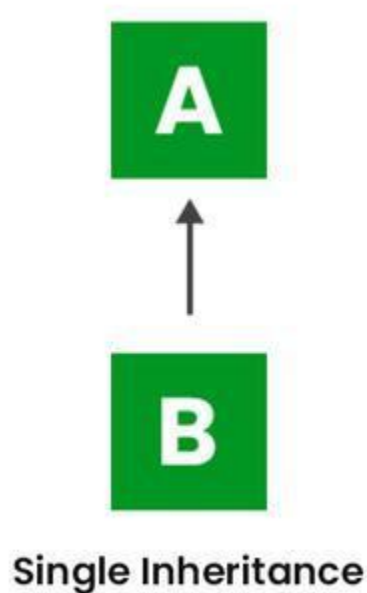
Types of Inheritance in Java

Below are the different types of inheritance which are supported by Java.

- Single Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance

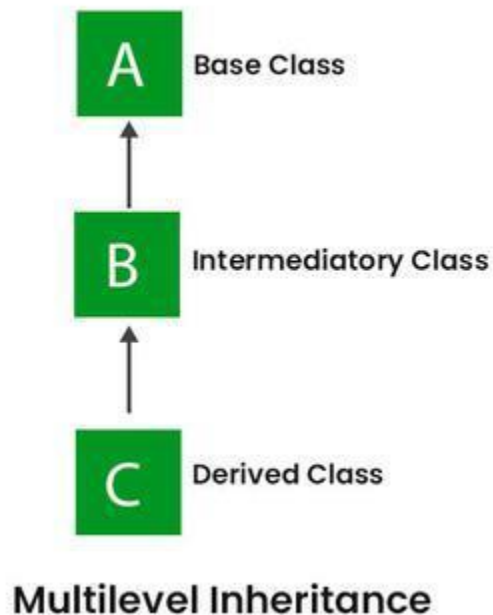
1. Single Inheritance

In single inheritance, a sub-class is derived from only one super class. It inherits the properties and behavior of a single-parent class. Sometimes, it is also known as simple inheritance.



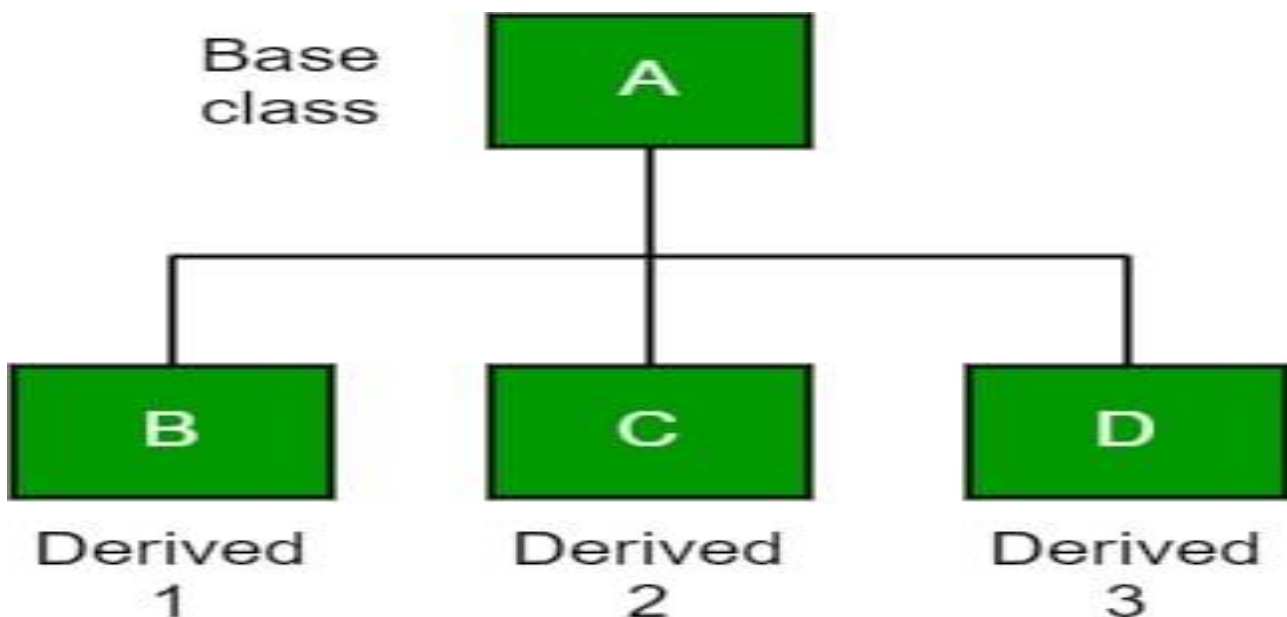
2. Multilevel Inheritance

In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes.



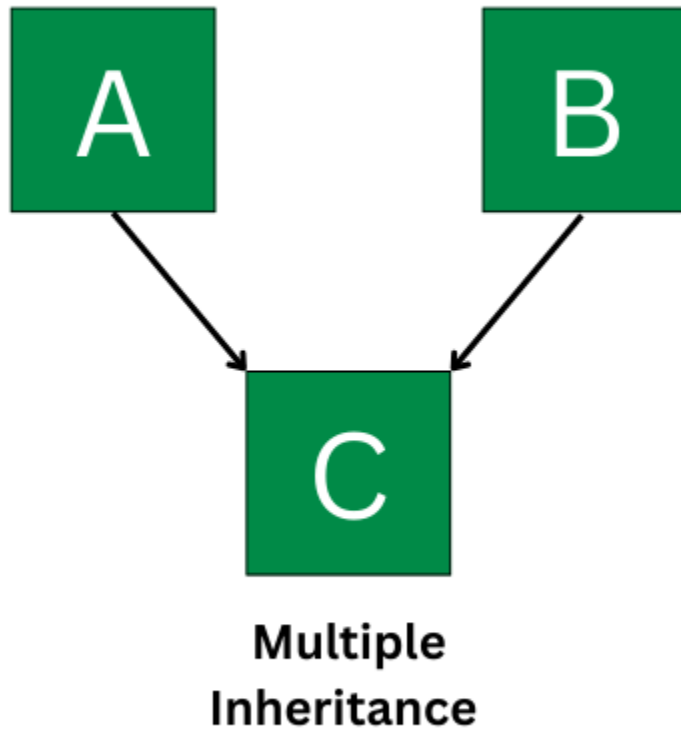
3. Hierarchical Inheritance

In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass.



4. Multiple Inheritance (Through Interfaces)

In multiple inheritance, one class can have more than one superclass and inherit features from all parent classes. Java does not support multiple inheritance with classes. In Java, we can achieve multiple inheritances only through interfaces.



Q.N. 2) Explain following keywords:

i) Extend

ii) super

➔ In java, the extends keyword is used to inherit all the properties and methods of the parent class. So basically, extends keyword is used to extend the functionality of the parent class to the subclass.

Example:

```
class One {  
    public void methodOne()  
    {  
    }  
}
```

```
class Two extends One {  
  
    public static void main(String args[])  
    {  
        Two t = new Two();  
        t.methodOne();  
    }  
}
```

ii) super

The super keyword in Java is a reference variable that is used to refer to the parent class when we are working with objects.

```
class Vehicle {  
    int maxSpeed = 120;  
}
```

```
class Car extends Vehicle {  
    int maxSpeed = 180;  
  
    void display()  
    {  
        System.out.println("Maximum Speed: "  
            + super.maxSpeed);  
    }  
}
```

```
class Test {  
    public static void main(String[] args)  
    {  
        Car small = new Car();  
        small.display();  
    }  
}
```

```
    }  
}
```

Q.N. 3) what is method overriding?

-> When a subclass provides its own version of a method that is already defined in its superclass, we call it method overriding. Overriding in Java occurs when a subclass or child class implements a method that is already defined in the superclass or base class.

Example:

```
class Animal {  
    void move() { System.out.println(  
        "Animal is moving.");  
    }  
    void eat() {  
        System.out.println(  
            "Animal is eating.");  
    }  
}  
  
class Dog extends Animal {  
    @Override void move()  
    {  
        System.out.println("Dog is running.");  
    }  
    void bark() {  
        System.out.println("Dog is barking.");  
    }  
}
```

```
}
```

```
public class Geeks {  
    public static void main(String[] args)  
    {  
        Dog d = new Dog();  
        d.move();  
        d.eat();  
        d.bark();  
    }  
}
```

Q.N. 4) What is package? WAP to create a package on your own.

-> Package in Java is a mechanism to encapsulate a group of classes, sub-packages, and interfaces. A package is a container of a group of related classes where some classes are accessible or exposed and others are kept for internal purposes.

All we need to do is put related classes into packages. After that, we can simply write an import class from existing packages and use it in our program. These packages fall into two categories: built-in packages and user-defined packages. Built-in packages encompass a wide range of functionalities such as java, lang, awt, javax, swing, net, io, util, SQL, etc. User-defined packages in Java offer the advantage of structured organization and enhanced maintenance.

// Save as Rectangle.java inside the "geometry" directory

```
package geometry;
```

```
public class Rectangle {  
    private double length;  
    private double width;
```

```

public Rectangle(double length, double width) {
    this.length = length;
    this.width = width;
}

public double calculateArea() {
    return length * width;
}

public double calculatePerimeter() {
    return 2 * (length + width);
}
}

// Save as Main.java in the parent directory

public class Main {
    public static void main(String[] args) {
        geometry.Rectangle rectangle = new geometry.Rectangle(5, 3);
        System.out.println("Area of the rectangle: " + rectangle.calculateArea());
        System.out.println("Perimeter of the rectangle: " + rectangle.calculatePerimeter());
    }
}

```

Q.N. 5) what is abstraction? Explain with example.

-> Abstraction in Java is the process of hiding the implementation details and only showing the essential details or features to the user. It allows to focus on what an object does rather than how it does it.

Example:

```
abstract class Geeks {  
    abstract void turnOn();  
    abstract void turnOff();  
}
```

```
class TVRemote extends Geeks {  
    @Override  
    void turnOn() {  
        System.out.println("TV is turned ON.");  
    }  
}
```

```
    @Override  
    void turnOff() {  
        System.out.println("TV is turned OFF.");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Geeks remote = new TVRemote();  
        remote.turnOn();  
        remote.turnOff();  
    }  
}
```

Q.N.6) What is the difference between abstraction and interface?

-> Difference Between Abstract Class and Interface:

Points	Abstract Class	Interface
Definition	Cannot be instantiated; contains both abstract (without implementation) and concrete methods (with implementation)	Specifies a set of methods a class must implement; methods are abstract by default.
Implementation Method	Can have both implemented and abstract methods.	Methods are abstract by default; Java 8, can have default and static methods.
Inheritance	class can inherit from only one abstract class.	A class can implement multiple interfaces.
Access Modifiers	Methods and properties can have any access modifier (public, protected, private).	Methods and properties are implicitly public.
Variables	Can have member variables (final, non-final, static, non-static).	Variables are implicitly public, static, and final (constants).

Q.N. 7) What is an error? Explain different types of error.

-> In computer programming, an error is an unplanned or unwanted program behavior that leads to the failure of the program or generates erroneous results.

Types of errors:

i) Compile time error

Compile Time Error

Compile Time Errors are those errors which prevent the code from running because of an incorrect syntax such as a missing semicolon at the end of a statement or a missing bracket, class not found, etc. These errors are detected by the java compiler and an error message is displayed on the screen while compiling. Compile Time Errors are sometimes also referred to as Syntax errors

```
class MisspelledVar {  
    public static void main(String args[])  
    {  
        int a = 40, b = 60;  
  
        // Declared variable Sum with Capital S  
        int Sum = a + b;  
  
        // Trying to call variable Sum  
        // with a small s ie. sum  
        System.out.println(  
            "Sum of variables is "  
            + sum);  
    }  
}
```

ii) Run time error

Run Time errors occur or we can say, are detected during the execution of the program. Sometimes these are discovered when the user enters an invalid data or data which is not relevant. Runtime errors occur when a program does not contain any syntax errors but asks the computer to do something that the computer is unable to reliably do.

```
// Java program to demonstrate Runtime Error
```

```
class DivByZero {  
    public static void main(String args[])  
    {  
        int var1 = 15;  
        int var2 = 5;  
        int var3 = 0;  
        int ans1 = var1 / var2;  
  
        // This statement causes a runtime error,  
        // as 15 is getting divided by 0 here  
        int ans2 = var1 / var3;  
  
        System.out.println(  
            "Division of va1"  
            + " by var2 is: "  
            + ans1);  
        System.out.println(  
            "Division of va1"  
            + " by var3 is: "  
            + ans2);  
    }  
}
```

Q.N. 8) Explain an exception? (Any 5).

-> An unexpected unwanted event that disturbs the program's normal execution after getting compiled while running is called an exception.

1) ArithmeticException

ArithmeticException can be defined as a runtime exception that can occur during an arithmetic operation due to the exceptional condition. ArithmeticException is part of the java.lang package.

```
class ArithmeticException_Demo
```

```
{  
    public static void main(String args[])  
    {  
        try {  
            int a = 30, b = 0;  
            int c = a/b; // cannot divide by zero  
            System.out.println ("Result = " + c);  
        }  
        catch(ArithmeticException e) {  
            System.out.println ("Can't divide a number by 0");  
        }  
    }  
}
```

2) Null Pointer Exception

A NullPointerException in Java is a Runtime Exception. In Java, a special null value can be assigned to an object reference. NullPointerException is thrown when a program attempts to use an object reference that has the null value.

```
class NullPointerException_Demo
```

```
{
```

```

public static void main(String args[])
{
    try {
        String a = null; //null value
        System.out.println(a.charAt(0));
    } catch(NullPointerException e) {
        System.out.println("NullPointerException..");
    }
}
}

```

3)StringIndexOutOfBoundsException

The StringIndexOutOfBoundsException in Java is a runtime exception that occurs when you try to access a character at an invalid index in a string. This can happen if the index is negative, equal to the string's length, or greater than the string's length.

```

class StringIndexOutOfBounds_Demo
{
    public static void main(String args[])
    {
        try {
            String a = "This is like chipping "; // length is 22
            char c = a.charAt(24); // accessing 25th element
            System.out.println(c);
        }
        catch(StringIndexOutOfBoundsException e) {
            System.out.println("StringIndexOutOfBoundsException");
        }
    }
}

```

```
}  
}
```

4) FileNotFoundException

To use `FileNotFoundException` in Java, you need to import the `java.io` package. This exception is thrown when an attempt to open the file denoted by a specified pathname has failed.

```
import java.io.File;  
  
import java.io.FileNotFoundException;  
  
import java.io.FileReader;  
  
class File_notFound_Demo {  
  
    public static void main(String args[]) {  
        try {  
  
            // Following file does not exist  
  
            File file = new File("E://file.txt");  
  
            FileReader fr = new FileReader(file);  
        } catch (FileNotFoundException e) {  
            System.out.println("File does not exist");  
        }  
    }  
}
```

5) NumberFormatException

`NumberFormatException` is a `RuntimeException` in Java that occurs when an attempt is made to convert a string with an improper format into a numeric value. This exception is a subclass of `IllegalArgumentException` and is thrown when it is not possible to convert a string into any numeric type such as `int`, `float`, `double`, etc.

```

class NumberFormat_Demo
{
    public static void main(String args[])
    {
        try {
            // "akki" is not a number
            int num = Integer.parseInt ("akki") ;

            System.out.println(num);
        } catch(NumberFormatException e) {
            System.out.println("Number format exception");
        }
    }
}

```

Q.N. 9) Explain try, catch and finally.

-> i) try

The try block contains a set of statements where an exception can occur.

It's where you place the code that you want to monitor for errors.

```

try {
    // Code that might throw an exception
    int result = 10 / 0; // This will throw an ArithmeticException
}

```

ii) catch

The catch block is used to handle the uncertain condition of a try block. A try block is always followed by a catch block, which handles the exception that occurs in the associated try block. You can have multiple catch blocks to handle different types of exceptions.

```
catch (ArithmeticException e)
{
    // Code to handle the exception

    System.out.println("ArithmeticException caught: " + e.getMessage());
}
```

iii) finally

The finally block contains code that will always execute, regardless of whether an exception was thrown or not. It's typically used for cleanup activities, like closing files or releasing resources.

```
finally {
    // Code that will always execute

    System.out.println("This will always execute.");
}
```

Example

```
public class ExceptionHandlingExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0; // This will throw an ArithmeticException
        } catch (ArithmeticException e) {
            System.out.println("ArithmeticException caught: " + e.getMessage());
        } finally {
            System.out.println("This will always execute.");
        }
    }
}
```

}

Q.N. 10) What is the use of throw Keyword?

-> The throw keyword in Java is used to explicitly throw an exception from a method or any block of code. Used within a method to actually throw an exception. It's followed by an instance of Throwable class (or its subclasses).

exception. It's followed by an instance of Throwable class (or its subclasses).

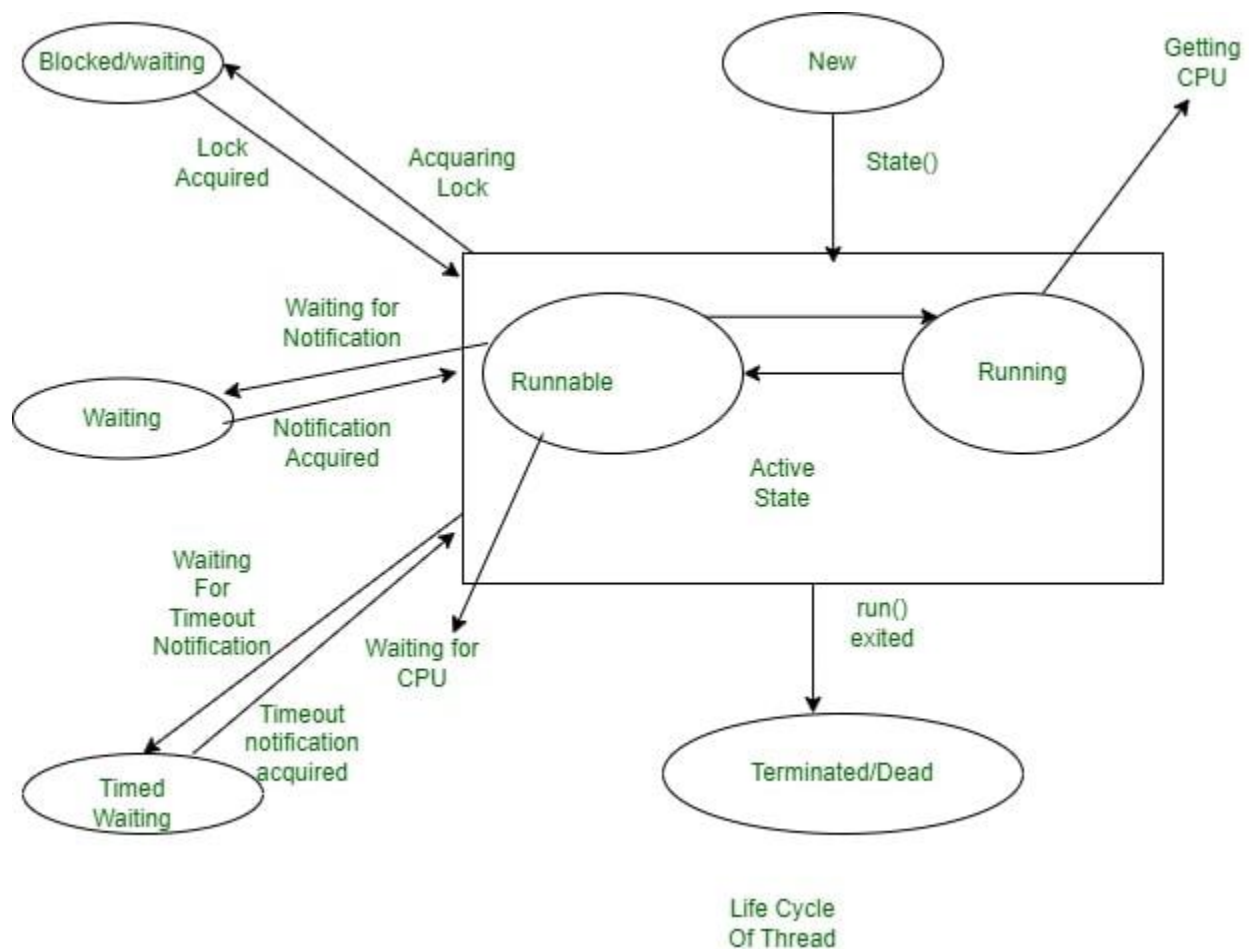
Example:

```
public void checkAge(int age) throws ArithmeticException {  
    if (age < 18) {  
        throw new ArithmeticException("Access denied - You must be at least 18 years old.");  
    } else {  
        System.out.println("Access granted - You are old enough!");  
    }  
}
```

Q.N. 11) What is Thread? What is the process of creating Thread?

-> Threads are lightweight subprocesses, representing the smallest unit of execution with separate paths. It allows multiple tasks to run in parallel within the same application, making programs more efficient by utilizing system resources more effectively. The process of thread or life cycle of thread are as follows:

1. New State
2. Active State
3. Waiting/Blocked State
4. Timed Waiting State
5. Terminated State



1. New State: By default, a Thread will be in a new state, in this state, code has not yet been run and the execution process is not yet initiated.

2. Active State: A Thread that is a new state by default gets transferred to Active state when it invokes the start() method, his Active state contains two sub-states namely:

- Runnable State: In This State, The Thread is ready to run at any given time and it's the job of the Thread Scheduler to provide the thread time for the runnable state preserved threads.
- Running State: When The Thread Receives CPU allocated by Thread Scheduler, it transfers from the "Runnable" state to the "Running" state. and after the expiry of its given time slice session, it again moves back to the "Runnable" state and waits for its next time slice.

3. Waiting/Blocked State: If a Thread is inactive but on a temporary time, then either it is a waiting or blocked state, for example, if there are two threads, T1 and T2 where T1 needs to communicate to the camera and the other thread T2 already using a camera to scan then T1 waits until T2

Thread completes its work, at this state T1 is parked in waiting for the state, and in another scenario, the user called two Threads T2 and T3 with the same functionality and both had same time slice given by Thread Scheduler then both Threads T1, T2 is in a blocked state. When there are multiple threads parked in a Blocked/Waiting state Thread Scheduler clears Queue by rejecting unwanted Threads and allocating CPU on a priority basis.

4. Timed Waiting State: Sometimes the longer duration of waiting for threads causes starvation, if we take an example like there are two threads T1, T2 waiting for CPU and T1 is undergoing a Critical Coding operation and if it does not exist the CPU until its operation gets executed then T2 will be exposed to longer waiting with undetermined certainty, In order to avoid this starvation situation, we had Timed Waiting for the state to avoid that kind of scenario as in Timed Waiting, each thread has a time period for which sleep() method is invoked and after the time expires the Threads starts executing its task.

5. Terminated State: A thread will be in Terminated State, due to the below reasons:

- Termination is achieved by a Thread when it finishes its task Normally.
- Sometimes Threads may be terminated due to unusual events like segmentation faults, exceptions...etc. and such kind of Termination can be called Abnormal Termination.
- A terminated Thread means it is dead and no longer available.

Q.N.12) How do you create Thread? Explain both by extending Thread and by implementing runnable interfaces.

-> In Java, you can create threads using two primary methods:

1. By Extending the Thread Class

You can create a custom thread by extending the Thread class and overriding its run() method. Here's how:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread is running...");  
    }  
}
```



```

public class Main {
    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // Create a new thread
        t1.start(); // Start the thread
    }
}

```

2. By Implementing the Runnable Interface

You can also create threads by implementing Runnable, which provides better flexibility (especially for multiple inheritance). Here's an example:

```

class MyRunnable implements Runnable {
    public void run() {
        System.out.println("Runnable thread is running...");
    }
}

public class Main {
    public static void main(String[] args) {
        MyRunnable myRunnable = new MyRunnable();
        Thread t1 = new Thread(myRunnable); // Pass Runnable instance to Thread
        t1.start();
    }
}

```

Q.N. 13) Why interthread communication is important? Explain.

->Inter-thread communication in Java is a mechanism in which a thread is paused from running in its critical section, and another thread is allowed to enter (or lock) the same critical section to be executed. Inter-thread communication in Java is a mechanism that allows synchronized

threads to communicate with each other. This is essential in scenarios where multiple threads need to coordinate their actions, such as in a producer-consumer problem. Java provides three primary methods for inter-thread communication: `wait()`, `notify()`, and `notifyAll()`. These methods are part of the `Object` class and must be used within a synchronized block.

Key Methods for Inter-Thread Communication

- `wait()`: This method causes the current thread to release the lock and wait until another thread invokes the `notify()` or `notifyAll()` method for the same object. It can also be used with a timeout to wait for a specified amount of time.

`public final void wait() throws InterruptedException`

`public final void wait(long timeout) throws InterruptedException`

- `notify()`: This method wakes up a single thread that is waiting on the object's monitor. The choice of which thread to wake up is arbitrary.

`public final void notify()`

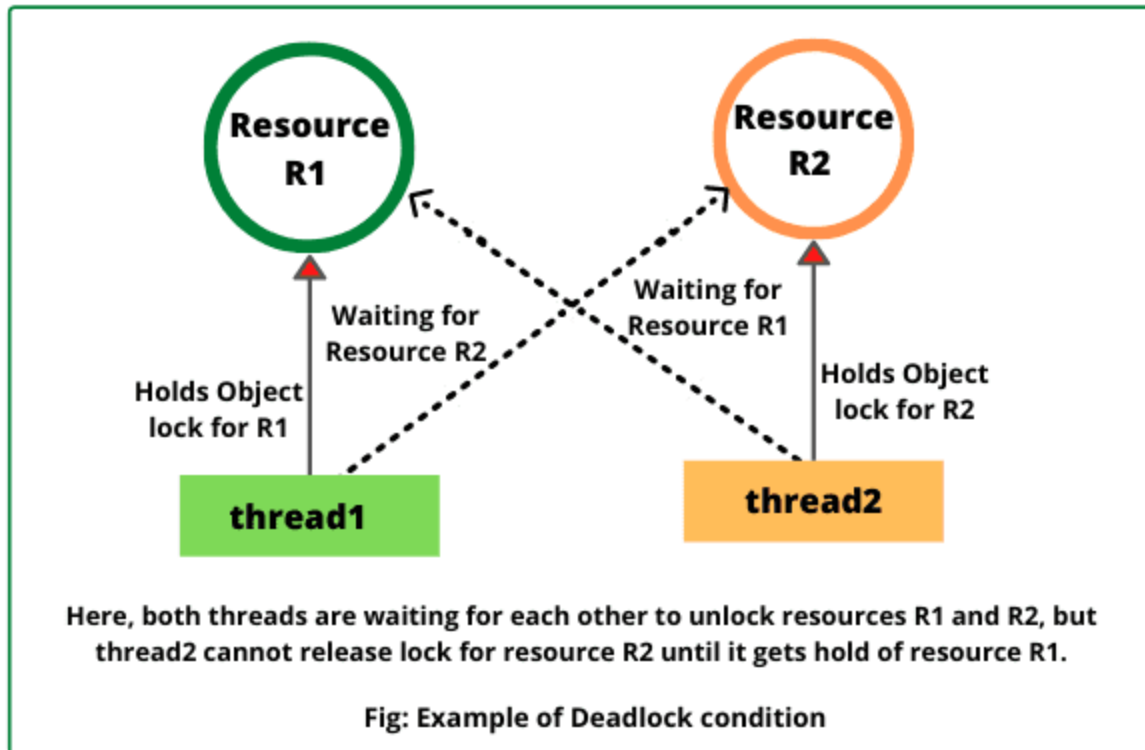
- `notifyAll()`: This method wakes up all the threads that are waiting on the object's monitor.

`public final void notifyAll()`

Q.N. 14) What is deadlock? What are the conditions of deadlock?

->Deadlock in Java is a situation that occurs when a thread is waiting for an object lock, that is acquired by another thread, and second thread is waiting for an object lock that is acquired by the first thread.

Since both threads are waiting for each other to release the lock simultaneously, this condition is called deadlock in Java. The object locks acquired by both threads are not released until their execution is not completed.



Deadlock in a program may occur due to the following conditions. They are as follows:

1. Mutual Exclusion: There is at least one resource that must be held in a non-sharable mode and hence, can be used only by one thread. If another thread requests for it, it should wait until the resource is available.
2. Hold and Wait: This condition occurs when one thread holds a resource and waits for another resource that is held by another thread.
3. No Preemption: The resource will be released only after the execution of thread is completed.
4. Circular Wait: This condition occurs when each thread is waiting for a resource held by the preceding one and the last thread is waiting for a resource held by first thread.

This is called circular wait deadlock. This is because every thread is waiting for a resource held by the next one and the last thread is waiting for a resource held by first.

A deadlock in a program can be prevented if any of the four conditions are not met. They are:

1. Mutual Exclusive Condition: If every resource is shared by multiple threads, deadlock would never occur.

2. Hold and Wait Condition: This condition can be eliminated when a thread is prohibited to wait for more resources while already holding a certain resource. It can be achieved when we declare all resources at the very beginning that are expected to use by a thread.

3. No Preemption Condition: This condition can be eliminated if a thread holding a certain resource is denied for further request. That thread must unlock its original resource. If necessary, request them again together with additional resource.

4. Circular Wait Condition: This is the easiest way to avoid deadlock than the above three. There are two ways to eliminate deadlock

Q.N. 15) What is synchronization? What are the advantages of synchronization? Explain with example.

-> In Java, synchronization is a mechanism that ensures that only one thread can access a resource at any given time. without Synchronization, data inconsistency or corruption can occur when multiple threads try to access and modify shared variables simultaneously. It helps in maintaining thread safety by preventing race conditions, data corruption, and unpredictable behavior.

Advantages of Synchronization

1. Thread Safety – Prevents multiple threads from modifying shared resources at the same time, avoiding data inconsistency.
2. Avoids Race Conditions – Ensures that critical sections of code are executed one thread at a time.
3. Reliable Execution – Synchronization makes sure that shared data is accessed in a predictable manner.
4. Consistency of Data – Ensures that all threads see the correct and updated values of shared variables.

Example:

```
class SharedResource {  
    private int count = 0;
```

```
    // Synchronized method to ensure thread safety
```

```
synchronized void increment() {  
    count++;  
    System.out.println(Thread.currentThread().getName() + " Count: " + count);  
}  
}
```

```
class MyThread extends Thread {  
    SharedResource resource;  
  
    MyThread(SharedResource resource) {  
        this.resource = resource;  
    }  
  
    public void run() {  
        for (int i = 0; i < 5; i++) {  
            resource.increment();  
        }  
    }  
}
```

```
public class SynchronizationExample {  
    public static void main(String[] args) {  
        SharedResource sharedResource = new SharedResource();  
  
        // Creating multiple threads that operate on the same resource  
        MyThread t1 = new MyThread(sharedResource);
```

```
        MyThread t2 = new MyThread(sharedResource);

        t1.start();

        t2.start();

    }

}
```

21. WAP to find factorial, prime or not, fibonacci series, queue?

=> 1. Factorial

```
public class Factorial {

    public static void main(String[] args) {

        int n = 5, fact = 1;

        for (int i = 1; i <= n; i++) {

            fact *= i;

        }

        System.out.println("Factorial of " + n + " = " + fact);

    }

}
```

2. Prime or Not

```
public class PrimeCheck {

    public static void main(String[] args) {

        int n = 5, count = 0;
```

```

for (int i = 2; i <= Math.sqrt(n); i++) {
    if (n % i == 0) {
        count++;
        break;
    }
}
if (n < 2 || count > 0)
    System.out.println(n + " is not prime");
else
    System.out.println(n + " is prime");
}
}

```

3.Fibonacci Series

```

public class Fibonacci {
    public static void main(String[] args) {
        int n = 5, a = 0, b = 1;
        System.out.print("Fibonacci Series: ");
        for (int i = 0; i < n; i++) {
            System.out.print(a + " ");
            int temp = a + b;
            a = b;
            b = temp;
        }
    }
}

```

```
}
```

4. Queue in Java using LinkedList

```
import java.util.*;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.add(10);
        q.add(20);
        System.out.println("Queue: " + q);
        System.out.println("Dequeued: " + q.poll());
        System.out.println("Queue after dequeue: " + q);
    }
}
```

22. What is layout Management. Explain flow layout, Greed Layout and Border Layout.

=> Layout Management in Java refers to the way components (like buttons, text fields, etc.) are arranged in a container (like a JFrame or JPanel). Java provides Layout Managers to handle this automatically, ensuring the UI adapts to screen size and resolution.

The common layout managers are:

1. FlowLayout

- Arranges components in a row, left to right.
- Moves to the next line when space runs out.
- Default layout for JPanel.

Example:

```
import javax.swing.*;
import java.awt.*;

public class FlowLayoutExample {
    public static void main(String[] args) {
        JFrame f = new JFrame("FlowLayout");
        f.setLayout(new FlowLayout());

        f.add(new JButton("A"));
        f.add(new JButton("B"));
        f.add(new JButton("C"));

        f.setSize(200, 150);
        f.setVisible(true);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

2. GridLayout

- Arranges components in a grid of rows and columns.
- All cells are of equal size.

Example:

```

import javax.swing.*;

import java.awt.*;

public class GridLayoutExample {

    public static void main(String[] args) {

        JFrame f = new JFrame("GridLayout");

        f.setLayout(new GridLayout(2, 2)); // 2 rows, 2 columns

        f.add(new JButton("1"));

        f.add(new JButton("2"));

        f.add(new JButton("3"));

        f.add(new JButton("4"));

        f.setSize(200, 200);

        f.setVisible(true);

        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    }

}

```

3. BorderLayout

- Divides container into 5 regions:
- NORTH, SOUTH, EAST, WEST, and CENTER
- Each region can hold only one component.

Example:

```

import javax.swing.*;

import java.awt.*;

public class BorderLayoutExample {

    public static void main(String[] args) {

        JFrame f = new JFrame("BorderLayout");

        f.setLayout(new BorderLayout());


        f.add(new JButton("North"), BorderLayout.NORTH);
        f.add(new JButton("South"), BorderLayout.SOUTH);
        f.add(new JButton("East"), BorderLayout.EAST);
        f.add(new JButton("West"), BorderLayout.WEST);
        f.add(new JButton("Center"), BorderLayout.CENTER);


        f.setSize(300, 200);

        f.setVisible(true);

        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

23. WAP to save data in database.

=> MySQL database:

```
CREATE DATABASE testdb;
```

```
USE testdb;
```

```
CREATE TABLE users (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(100));
```

```
//Insert data into database
```

```
import java.sql.*;
```

```
public class SaveToDatabase {
```

```
    public static void main(String[] args) {
```

```
        String url = "jdbc:mysql://localhost:3306/testdb"; // DB URL
```

```
        String user = "root"; // your DB username
```

```
        String pass = ""; // your DB password
```

```
        try {
```

```
            Connection con = DriverManager.getConnection(url, user, pass);
```

```
            String query = "INSERT INTO users (name) VALUES (?)";
```

```
            PreparedStatement ps = con.prepareStatement(query);
```

```
            ps.setString(1, "John Doe"); // value to insert
```

```
            ps.executeUpdate();
```

```
            System.out.println("Data inserted successfully.");
```

```
            con.close();
```

```
        } catch (Exception e) {
```

```
            System.out.println("Error: " + e);
```

```
        }
```

```
    }
```

```
}
```

24. What is applet? What is the lifecycle of applet.

=> An applet is a small Java program that runs inside a web browser or an applet viewer. - It is used for creating interactive web applications.

Applets are subclasses of `java.applet.Applet` or `javax.swing.JApplet`.

Applet Lifecycle in Java

-The lifecycle of an applet is controlled by the browser or applet viewer, which calls the following methods:

Lifecycle Methods:

1.init()

- Called once when the applet is first loaded.
- Used for initialization like setting up UI components.

2.start()

- Called each time the applet becomes active (visible).
- Can be called multiple times.

3.paint(Graphics g)

- Called to draw the applet's output.
- Also called when the window is resized or refreshed.

4.stop()

- Called when the applet becomes inactive or user navigates away.

-Used to pause animations, etc.

5.destroy()

-Called when the applet is removed from memory.

-Used for cleanup like closing connections.

Applet Lifecycle Diagram:

fig yeta rakha

25. WAP to show Hello World, Line, circle, Rectangle by using applet.

=>

1. Display "Hello World" in Applet

```
import java.applet.Applet;
```

```
import java.awt.Graphics;
```

```
public class HelloWorldApplet extends Applet {  
    public void paint(Graphics g) {  
        g.drawString("Hello World", 20, 20); // Displays "Hello World"  
    }  
}
```

2. Draw a Line in Applet

```
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class LineApplet extends Applet {  
    public void paint(Graphics g) {  
        g.drawLine(20, 40, 120, 40); // Draws a line from (20,40) to (120,40)  
    }  
}
```

3. Draw a Rectangle in Applet

```
import java.applet.Applet;  
import java.awt.Graphics;  
  
public class RectangleApplet extends Applet {  
    public void paint(Graphics g) {  
        g.drawRect(20, 60, 100, 50); // Draws a rectangle at (20,60) with width 100 and height 50  
    }  
}
```

4. Draw a Circle in Applet

```
import java.applet.Applet;
```

```
import java.awt.Graphics;
```

```
public class CircleApplet extends Applet {
```

```
    public void paint(Graphics g) {
```

```
        g.drawOval(20, 130, 50, 50); // Draws a circle (ellipse with equal width and height)
```

```
    }
```

```
}
```